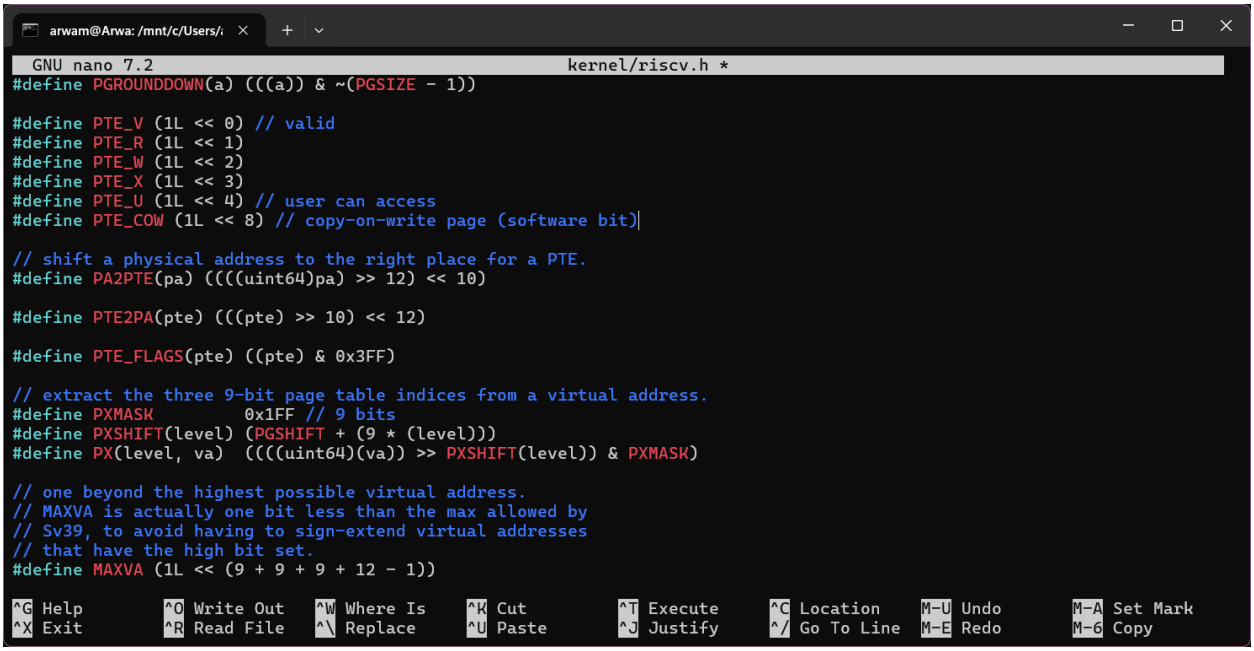
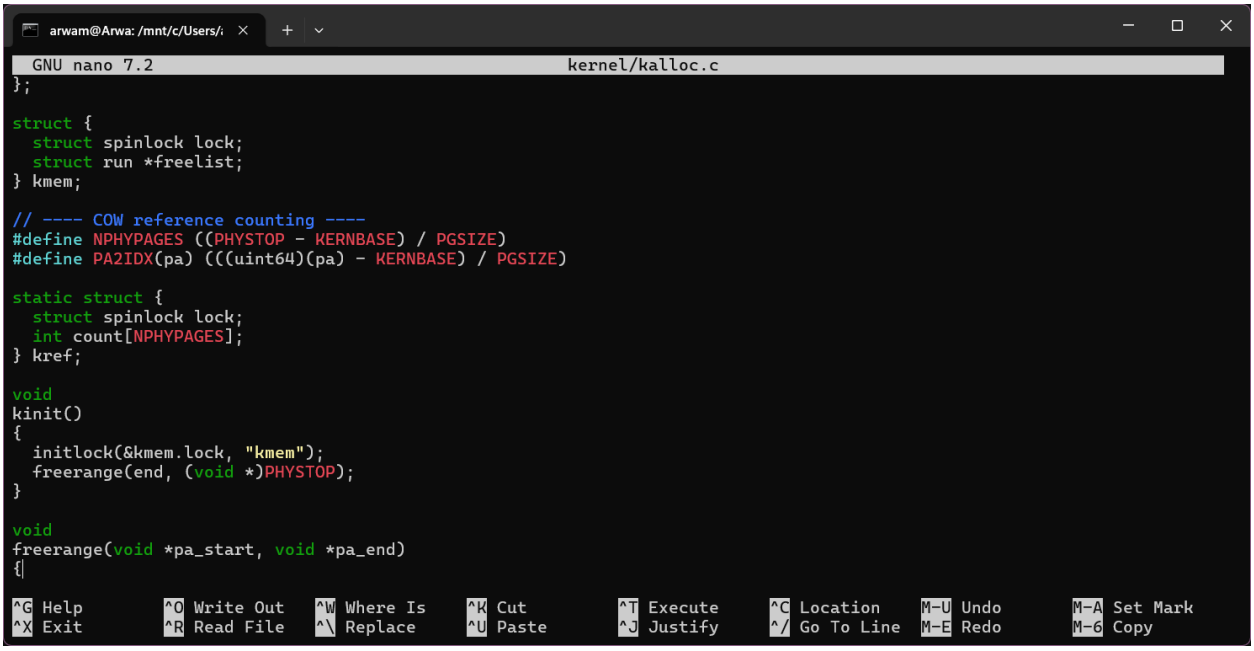

arwam@Arwa: /mnt/c/Users/a
GNU nano 7.2 kernel/riscv.h *
#define PGROUNDDOWN(a) (((a)) & ~(PGSIZE - 1))
#define PTE_V (1L << 0) // valid
#define PTE_R (1L << 1)
#define PTE_W (1L << 2)
#define PTE_X (1L << 3)
#define PTE_U (1L << 4) // user can access
#define PTE_COW (1L << 8) // copy-on-write page (software bit)
// shift a physical address to the right place for a PTE.
#define PA2PTE(pa) ((((uint64)pa) >> 12) << 10)
#define PTE2PA(pte) (((pte) >> 10) << 12)
#define PTE_FLAGS(pte) ((pte) & 0x3FF)
// extract the three 9-bit page table indices from a virtual address.
#define PXMASK 0x1FF // 9 bits
#define PXSHIFT(level) (PGSHIFT + (9 * (level)))
#define PX(level, va) ((((uint64)(va)) >> PXSHIFT(level)) & PXMASK)
// one beyond the highest possible virtual address.
// MAXVA is actually one bit less than the max allowed by
// Sv39, to avoid having to sign-extend virtual addresses
// that have the high bit set.
#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy


arwam@Arwa: /mnt/c/Users/a
GNU nano 7.2 kernel/kalloc.c
};
struct {
struct spinlock lock;
struct run *freelist;
} kmem;
// ---- COW reference counting ----
#define NPHYPAGES ((PHYSTOP - KERNBASE) / PGSIZE)
#define PA2IDX(pa) (((uint64)(pa) - KERNBASE) / PGSIZE)
static struct {
struct spinlock lock;
int count[NPHYPAGES];
} kref;
void
kinit()
{
initlock(&kmem.lock, "kmem");
freerange(end, (void *)PHYSTOP);
}
void
freerange(void *pa_start, void *pa_end)
{
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

arwam@Arwa: /mnt/c/Users/a

```
  GNU nano 7.2                                          kernel/kalloc.c *
static struct {
  struct spinlock lock;
  int count[NPHYPAGES];
} kref;

void
kinit()
{
  initlock(&kmem.lock, "kmem");
  initlock(&kref.lock, "kref");
  freerange(end, (void *)PHYSTOP);
}

void
freerange(void *pa_start, void *pa_end)
{
  char *p;
  p = (char *)PGROUNDUP((uint64)pa_start);
  for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
    kref.count[PA2IDX(p)] = 1;      // ← preset so kfree() drops it to 0
    kfree(p);
}

// Free the page of physical memory pointed at by pa,
// which normally should have been returned by a
// call to kalloc().  (The exception is when

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

arwam@Arwa: /mnt/c/Users/a

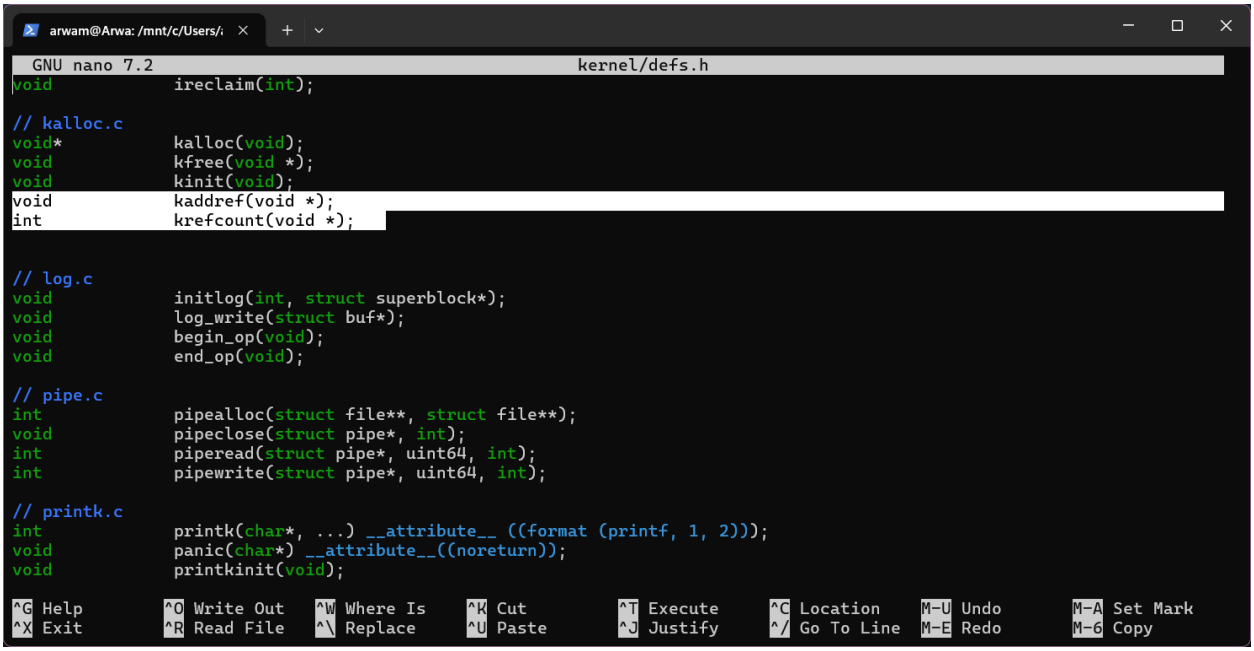
```
  GNU nano 7.2                                          kernel/kalloc.c *
  release(&kmem.lock);
}

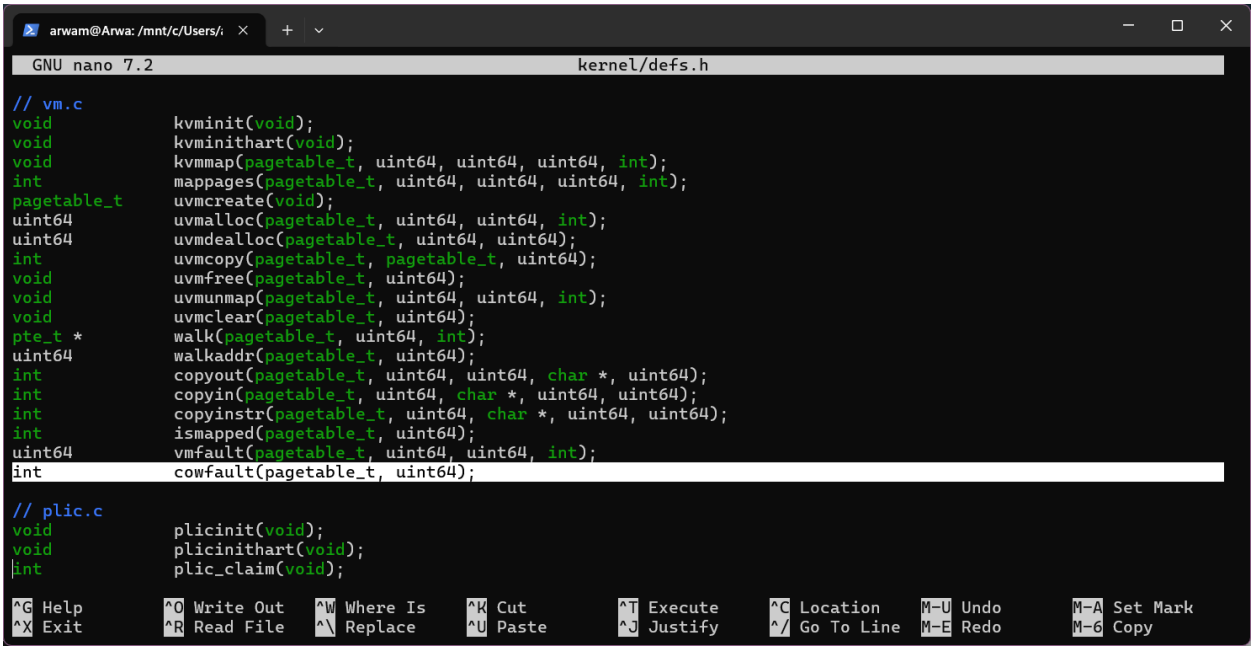
// Allocate one 4096-byte page of physical memory.
// Returns a pointer that the kernel can use.
// Returns 0 if the memory cannot be allocated.
void *
kalloc(void)
{
  struct run *r;

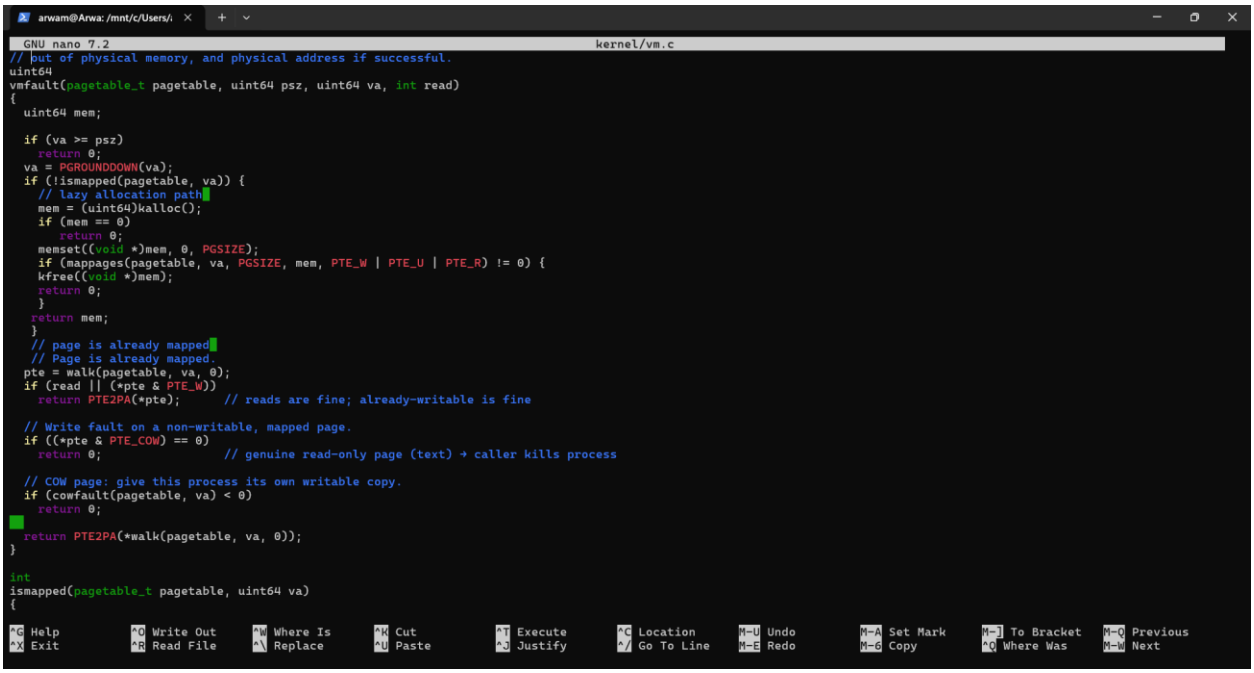
  acquire(&kmem.lock);
  r = kmem.freelist;
  if (r)
    kmem.freelist = r->next;
  release(&kmem.lock);

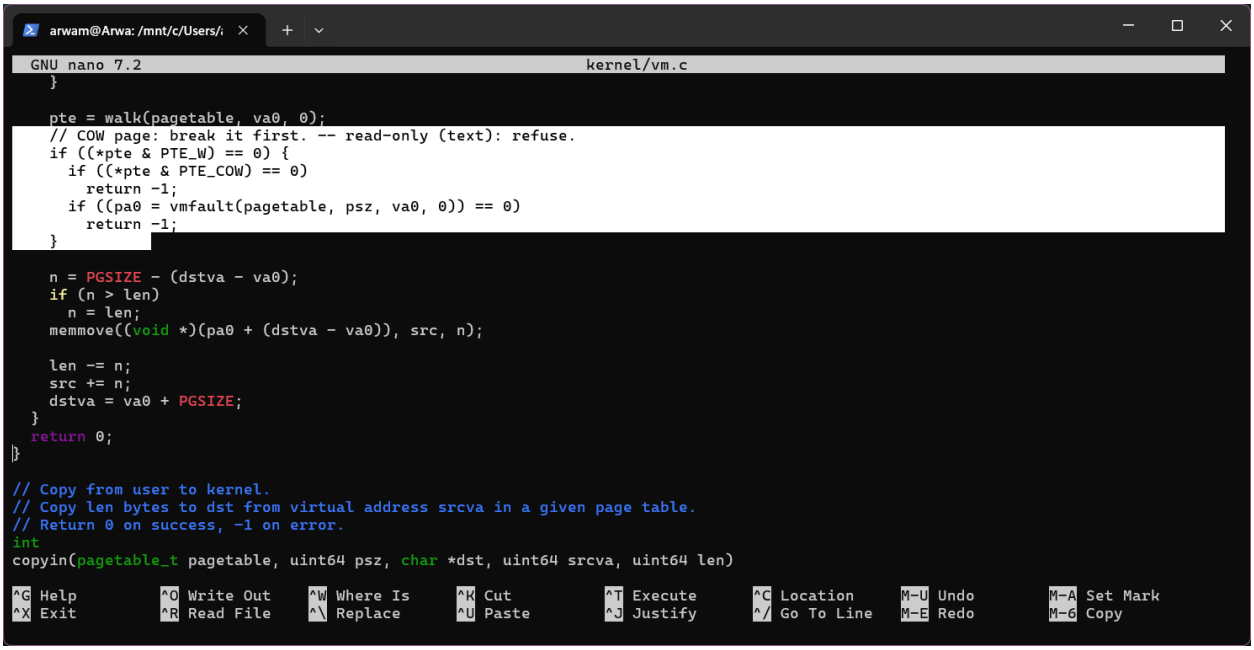
  if (r){
    memset((char *)r, 5, PGSIZE); // fill with junk
    acquire(&kref.lock);
    kref.count[PA2IDX(r)] = 1;      // fresh page: one owner
    release(&kref.lock);
  }
   return (void *)r;
}

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```


arwam@Arwa: /mnt/c/Users/a
GNU nano 7.2 kernel/defs.h
void ireclaim(int);
// kalloc.c
void* kalloc(void);
void kfree(void *);
void kinit(void);
void kaddref(void *);
int krefcount(void *);
// log.c
void initlog(int, struct superblock*);
void log_write(struct buf*);
void begin_op(void);
void end_op(void);
// pipe.c
int pipealloc(struct file**, struct file**);
void pipeclose(struct pipe*, int);
int piperead(struct pipe*, uint64, int);
int pipewrite(struct pipe*, uint64, int);
// printk.c
int printk(char*, ...) __attribute__ ((format (printf, 1, 2)));
void panic(char*) __attribute__((noreturn));
void printkinit(void);
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy


arwam@Arwa: /mnt/c/Users/a
GNU nano 7.2 kernel/defs.h
// vm.c
void kvminit(void);
void kvminithart(void);
void kvmmap(pagetable_t, uint64, uint64, uint64, int);
int mappages(pagetable_t, uint64, uint64, uint64, int);
pagetable_t uvmcreate(void);
uint64 uvmalloc(pagetable_t, uint64, uint64, int);
uint64 uvmdealloc(pagetable_t, uint64, uint64);
int uvmcopy(pagetable_t, pagetable_t, uint64);
void uvmfree(pagetable_t, uint64);
void uvmunmap(pagetable_t, uint64, uint64, int);
void uvmclear(pagetable_t, uint64);
pte_t * walk(pagetable_t, uint64, int);
uint64 walkaddr(pagetable_t, uint64);
int copyout(pagetable_t, uint64, uint64, char *, uint64);
int copyin(pagetable_t, uint64, char *, uint64, uint64);
int copyinstr(pagetable_t, uint64, char *, uint64, uint64);
int ismapped(pagetable_t, uint64);
uint64 vmfault(pagetable_t, uint64, uint64, int);
int cowfault(pagetable_t, uint64);
// plic.c
void plicinit(void);
void plicinithart(void);
int plic_claim(void);
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy


arwam@Arwa: /mnt/c/Users/
GNU nano 7.2 kernel/vm.c
// out of physical memory, and physical address if successful.
uint64
vmfault(pagetable_t pagetable, uint64 psz, uint64 va, int read)
{
uint64 mem;
if (va >= psz)
return 0;
va = PGROUNDDOWN(va);
if (!ismapped(pagetable, va)) {
// lazy allocation path
mem = (uint64)kalloc();
if (mem == 0)
return 0;
memset((void *)mem, 0, PGSIZE);
if (mappages(pagetable, va, PGSIZE, mem, PTE_W | PTE_U | PTE_R) != 0) {
kfree((void *)mem);
return 0;
}
return mem;
}
// page is already mapped
// Page is already mapped.
pte = walk(pagetable, va, 0);
if (read || (*pte & PTE_W))
return PTE2PA(*pte); // reads are fine; already-writable is fine
// Write fault on a non-writable, mapped page.
if ((*pte & PTE_COW) == 0)
return 0; // genuine read-only page (text) → caller kills process
// COW page: give this process its own writable copy.
if (cowfault(pagetable, va) < 0)
return 0;
return PTE2PA(*walk(pagetable, va, 0));
}
int
ismapped(pagetable_t pagetable, uint64 va)
{
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-Q Previous
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-W Next


arwam@Arwa: /mnt/c/Users/
GNU nano 7.2 kernel/vm.c
}
pte = walk(pagetable, va0, 0);
// COW page: break it first. -- read-only (text): refuse.
if ((*pte & PTE_W) == 0) {
if ((*pte & PTE_COW) == 0)
return -1;
if ((pa0 = vmfault(pagetable, psz, va0, 0)) == 0)
return -1;
}
n = PGSIZE - (dstva - va0);
if (n > len)
n = len;
memmove((void *)(pa0 + (dstva - va0)), src, n);
len -= n;
src += n;
dstva = va0 + PGSIZE;
}
return 0;
}
// Copy from user to kernel.
// Copy len bytes to dst from virtual address srcva in a given page table.
// Return 0 on success, -1 on error.
int
copyin(pagetable_t pagetable, uint64 psz, char *dst, uint64 srcva, uint64 len)
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

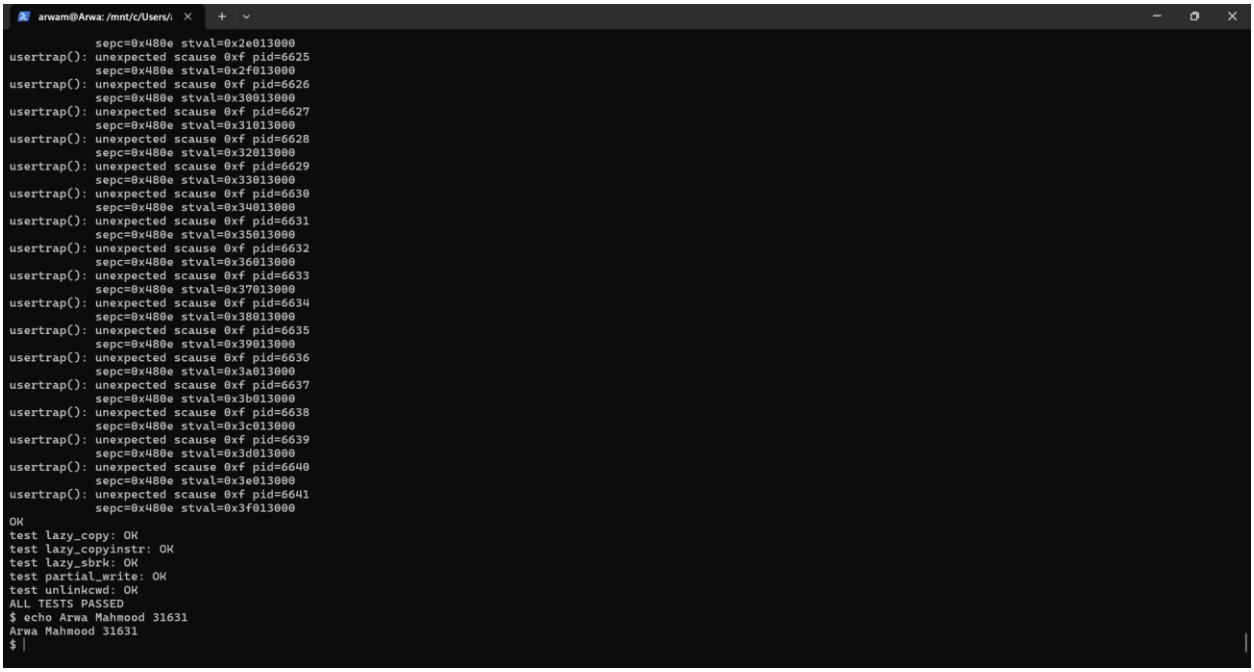
ALL TESTS PASSED!! (cowtest + usertests -q + grade)



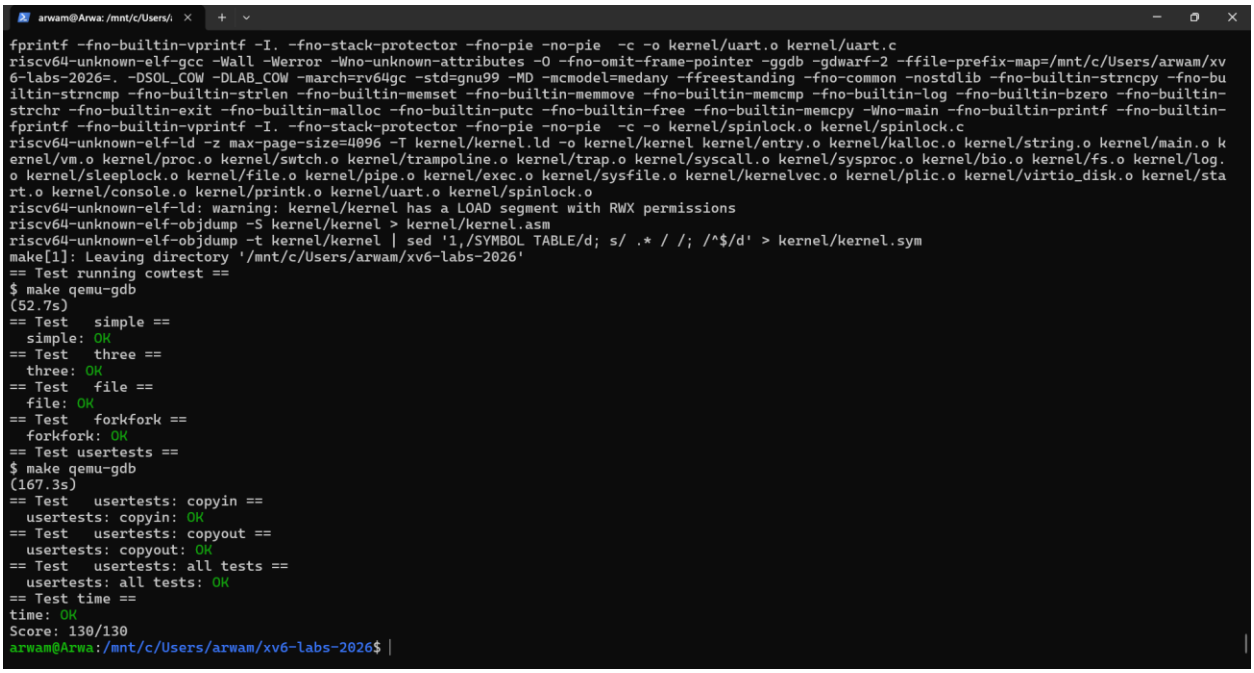
```
arwam@Arwa: /mnt/c/Users/
ee -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o user/dorphan.o user/dorph
an.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_dorphan user/dorphan.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_dorphan > user/dorphan.asm
riscv64-unknown-elf-objdump -t user/_dorphan | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/dorphan.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=. -DSOL_COW -D
LAB_COW -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memse
t -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-fr
ee -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o user/sync.o user/sync.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_sync user/sync.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_sync > user/sync.asm
riscv64-unknown-elf-objdump -t user/_sync | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/sync.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=. -DSOL_COW -D
LAB_COW -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memse
t -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-fr
ee -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie   -c -o user/cowtest.o user/cowte
st.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_cowtest user/cowtest.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_cowtest > user/cowtest.asm
riscv64-unknown-elf-objdump -t user/_cowtest | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/cowtest.sym
mkfs/mkfs fs.img README  user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertes
ts user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync  user/_cowtest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1015 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,
id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo arwa mahmood, 31631
arwa mahmood, 31631
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
forkfork: ok
ALL COW TESTS PASSED
$
```



```
arwam@Arwa: /mnt/c/Users/
          sepc=0x480e stval=0x2e013000
usertrap(): unexpected scause 0xf pid=6625
          sepc=0x480e stval=0x2f013000
usertrap(): unexpected scause 0xf pid=6626
          sepc=0x480e stval=0x30013000
usertrap(): unexpected scause 0xf pid=6627
          sepc=0x480e stval=0x31013000
usertrap(): unexpected scause 0xf pid=6628
          sepc=0x480e stval=0x32013000
usertrap(): unexpected scause 0xf pid=6629
          sepc=0x480e stval=0x33013000
usertrap(): unexpected scause 0xf pid=6630
          sepc=0x480e stval=0x34013000
usertrap(): unexpected scause 0xf pid=6631
          sepc=0x480e stval=0x35013000
usertrap(): unexpected scause 0xf pid=6632
          sepc=0x480e stval=0x36013000
usertrap(): unexpected scause 0xf pid=6633
          sepc=0x480e stval=0x37013000
usertrap(): unexpected scause 0xf pid=6634
          sepc=0x480e stval=0x38013000
usertrap(): unexpected scause 0xf pid=6635
          sepc=0x480e stval=0x39013000
usertrap(): unexpected scause 0xf pid=6636
          sepc=0x480e stval=0x3a013000
usertrap(): unexpected scause 0xf pid=6637
          sepc=0x480e stval=0x3b013000
usertrap(): unexpected scause 0xf pid=6638
          sepc=0x480e stval=0x3c013000
usertrap(): unexpected scause 0xf pid=6639
          sepc=0x480e stval=0x3d013000
usertrap(): unexpected scause 0xf pid=6640
          sepc=0x480e stval=0x3e013000
usertrap(): unexpected scause 0xf pid=6641
          sepc=0x480e stval=0x3f013000
OK
test lazy_copy: OK
test lazy_copyinstr: OK
test lazy_sbrk: OK
test partial_write: OK
test unlinkcwd: OK
ALL TESTS PASSED
$ echo Arwa Mahmood 31631
Arwa Mahmood 31631
$
```


arwam@Arwa: /mnt/c/Users/a
fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/uart.o kernel/uart.c
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/mnt/c/Users/arwam/xv6-labs-2026=. -DSOL_COW -DLAB_COW -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/spinlock.o kernel/spinlock.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/start.o kernel/console.o kernel/printk.o kernel/uart.o kernel/spinlock.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
make[1]: Leaving directory '/mnt/c/Users/arwam/xv6-labs-2026'
== Test running cowtest ==
$ make qemu-gdb
(52.7s)
== Test   simple ==
  simple: OK
== Test   three ==
  three: OK
== Test   file ==
  file: OK
== Test   forkfork ==
  forkfork: OK
== Test usertests ==
$ make qemu-gdb
(167.3s)
== Test   usertests: copyin ==
  usertests: copyin: OK
== Test   usertests: copyout ==
  usertests: copyout: OK
== Test   usertests: all tests ==
  usertests: all tests: OK
== Test time ==
time: OK
Score: 130/130
arwam@Arwa:/mnt/c/Users/arwam/xv6-labs-2026$